

花卉题解

说明： - 本题暂未完成选做题，在之后会逐渐完成； - 更多其他与代码相关的学习笔记，在py文件中以注释形式存在； - 完整的代码和此篇题解的PDF版放到了github仓库里，github仓库链接如下

附件：[github仓库链接](#))

II-初识卷积神经网络

一. 一张 RGB 三通道彩色图片是如何被处理成可用于训练的数据？张量又是什么？

- 1. RGB 三通道彩色图片首先被转化成张量（此时已经算得上是可用于训练的数据了），然后拆分成R、G、B三张矩阵图（三维变二维），接着再用卷积核进行卷积运算，进一步提取特征，转化成为更方便进行训练的数据。
- 2. 张量是三维的一种数据形式（从0维到2维都有特殊的名称，依次是数，向量，矩阵，张量），张量是一种方便计算机计算的一种数据，特别是能够让计算机进行并行运算和利用显卡计算。一张彩图的数据可以形成三维张量（也叫三阶张量）（高，宽，色彩），再拓展，可以用四阶张量表示一个包含多张图片的数据集（编号，高，宽，色彩）

三阶张量如下图：

	0	1	2	3	4	5	6	7	8	9	...	310	311	312	313	314	315	316	317	318	319
0	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	...	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]
1	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	...	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]
2	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	...	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]
3	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	...	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]
4	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	...	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]

二. 一个卷积神经网络分为哪几层？

- 卷积神经网络是一个训练模型的范式（卷积神经网络应该还算不上一个模型，因为其并没有确定使用的参数）。卷积神经网络常有这些层：输入层、卷积层、激活层、池化层、全连接层、输出层，以上为层的种类，具体的层数以具体的训练效果做参考。

激活层是否单独存在在不同文章中说法不同，也有的文章认为激活函数是附着于卷积层的工具，不单独分层

卷积层、激活层、池化层常组成为一个整体多层叠加，也就是（卷积层+激活层+池化层）*n，当然也有的时候不加池化层，输出层也常常不加激活层。

三.什么是卷积，数学严格意义上的卷积和神经网络中的卷积操作有区别吗？卷积核、步长是什么？

1. 卷积是一种数学运算，通过将两个函数翻转、平移并叠加乘积，生成一个新的函数，用于描述系统对输入的累积响应。
2. 有区别：最明显的是，神经网络里的卷积是没有经过翻转这一步的，是通过学习得到卷积核之后，直接惊醒平移并叠加乘积；其次神经网络里的卷积的两个函数是小范围的二阶张量。
3. 卷积核：是对图片张量进行特征提取的过滤器，也是一张小的、可学习的权重表，直接执行逐元素的乘法和加法，卷积核的不同会引起不同的后果，比如边缘淡化，锐化，突出某一特征等。（卷积核一般由训练获得，**靠反向传播 + 优化器训出来的**）
4. 步长：指卷积核一次平移的像素点数，一般会影响训练的精度和计算量。

四.池化有哪些类型，其作用分别有什么？感受野是什么？

1. 池化的类型及具体作用
2. 最大池化：取每个池化窗口的最大值，能提高模型鲁棒性，突出了显著特征
3. 平均池化：取每个池化窗口的平均值，能把随机噪声被平均掉，数据更稳
4. 重叠池化：相邻池化窗口有重叠部分，提高精度防止造成边缘附近数据相关性的遗漏，还可减弱过拟合（因为增强了一些些鲁棒性，不是因为增加了数据）
5. 空间金字塔池化：池化窗口分别取1, 2, 3*3.....,每个窗口取最大值；避免了为适应全连接层而对图像进行裁剪/伸缩造成的误差。
6. 随机池化：根据池化窗口内元素的值按随机选择，增强模型泛化能力，减少过拟合
7. 混合池化：结合最大池化和平均池化的优点，同时提取最大值和平均值，提取更丰富的特征信息
8. 全局池化：把高宽压到1，顶替全连接层、把参数量砍掉几个数量级，防止过拟合
9. 池化的共同作用：
降维，减小计算；增加鲁棒性（降低对部分噪音，大小变换，平移的敏感程度，因为无论有没有噪音，

池化会让他们变得类似)；以防过拟合(减少参数量)

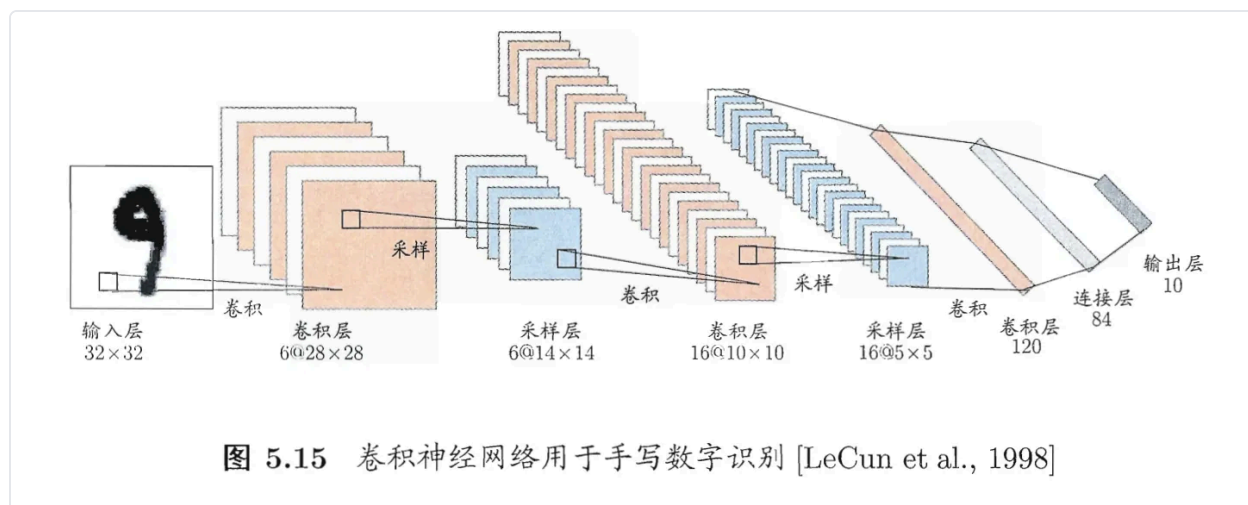
10. 感受野：每一层输出的特征图上某个像素点所对应的原始图像上的区域大小，第一层的输出特征图一个像素点对应的原始图像区域等于卷积核的大小，而其他卷积层输出特征图一个像素点对应的原始图像区域与之前所有层的卷积核大小和步长有关。简单来说，**感受野**就是之后的像素点包含或者简化的原始图像的面积。卷积+池化的组合能较快速的增大感受野。**

11. 理论感受野：通过卷积核大小，步长，层数计算的感受野大小。公式如下：

$$r_0 = \sum_{l=1}^L \left((k_l - 1) \prod_{i=1}^{l-1} s_i \right) + 1$$

12. 有效感受野：随着网络的加深，感受野外部的信息会丢失，且会有空隙，会不对称（这是因为权重随机初始化），可以通过将一个特征像素点反向传播看真正有贡献的原始图像有多大来看出有效感受野的大小。未训练的有效感受野成高斯分布，带有空隙。

四. 激活层在此的作用？最后卷积神经网络会输出什么？



1. 卷积操作是由输入矩阵与卷积核矩阵进行计算，本质上是 $WX+B$ (W, X, B 为矩阵)，如果不加激活层，多次卷积可以等效为一次，做的是无用功。激活层可以对卷积层的输出层进行非线性的映射，让输出的特征图具有非线性，增加模型复杂度，提高模型的性能。2. 最后卷积神经网络会输出分数（有正有负，且和不一定为0），可通过softmax转为概率，利于分类。（分数有正有负也是不能用输出层不能用relu的原因之一）

III-数据预处理

1. 数据集中存在异常或者损坏的图片无法打开（图片默认格式为jpg）：

通过两次识别判断是否出错，如果出错，将异常的图片移动到特定文件夹，并且在.txt文件中记录异常图片的来源

```
# preprocess.py===== 坏图检测 =====
def handle_bad_img(logs_dir=LOGS_DIR, bad_img=BAD_IMG, train_dir=TRAIN_DIR, test_dir=TEST_DIR):
    os.makedirs(logs_dir, exist_ok=True)
    bad_img = open(bad_img, 'w', encoding='utf-8')
    # 'w' 为每次清空重写，'a' 为追加记录；open() 的返回值必须用一个对象接住
    # encoding='utf-8': 写中文用哪套编码，读的时候也得用同一套，否则 Windows 用默认的 GBK 会乱
    码
    all_img_sort_dir = [test_dir, train_dir]
    bad = []
    bad_img.write('以下为坏图：\n')
    for img_sort_dir in all_img_sort_dir:
        for img_sort in os.listdir(img_sort_dir): # listdir 不接受列表
            img_sort_path = os.path.join(img_sort_dir, img_sort)
            if not os.path.isdir(img_sort_path): # 不是文件夹就跳过
                continue
            for img in os.listdir(img_sort_path): # 还要再进一层类别（sort）文件夹
                img_path = os.path.join(img_sort_path, img) # listdir 只返回文件名，必须拼回完
                整路径
                try:
                    img = Image.open(img_path)
                    img.verify() # 只读文件头，快，但抓不全所有坏图
                    img = Image.open(img_path)
                    img.load() # 真正解码像素，能抓出 verify 漏掉的坏图
                    result = True
                except Exception:
                    result = False
                if not result: # not result ,即 result 为 False 时进行
                    bad.append(img_path)
                    bad_img.write(img_path + '\n')
    bad_img.close()
    if bad != []:
        broken_dir = os.path.join(logs_dir, "broken")
        os.makedirs(broken_dir, exist_ok=True)
```

```

    for p in bad:
        # os.path.dirname() 往左砍掉最后一段，返回剩下的
        # os.path.basename() 往右取最后一段

        name = os.path.basename(os.path.dirname(os.path.dirname(p))) + "_" +
os.path.basename(os.path.dirname(p)) + "_" + os.path.basename(p)

        dst = os.path.join(broken_dir, name)

        if not os.path.exists(dst): # 目标位置没有同名文件才移动
            shutil.move(p, dst) # 剪切，不删除，随时可还原

    return bad

#main.py=====清理坏图=====
bad = handle_bad_img()
print("坏图", len(bad), "张")

```

1. 划分数据集:

下载的数据集自带train,test的分集，我将train中的数据分出了20%作为验证集（val），但是验证集的图片没有专门分出一个文件夹（没有改变硬盘数据），只是取用的时候train文件夹中的图片，一部分作为train，一部分作为val

```

# preprocess.py=====从训练集中划分验证集 =====
def val_split(dataset, val_size=VAL_SIZE, seed=SEED):
    n = len(dataset)
    n_val = int(round(n * val_size)) # random_split 的长度必须是整数
    n_train = n - n_val # 用减法保证 n_train + n_val == n, 不会漏样本
    gener = torch.Generator().manual_seed(seed) # 造一个单独的随机数生成器，不受全局种子影响
    train, val = random_split(dataset, [n_train, n_val], generator=gener)
    # 划分是逻辑划分：只记录下标，不复制图片、不新建 val 文件夹（硬盘数据不变）
    return train, val

#main.py=====建立dataset=====
train_dataset_tr = build_dataset("train", train_tf)
train_dataset_eval = build_dataset("train", eval_tf)#transform增强后的图根本不会被存下来 — 它是在取批次的那一瞬间现算现用、用完即弃的。
#以上两个其实一样，只是取用的时候取的数据一个要增强，一个不增强
test_dataset = build_dataset("test", eval_tf)

#main.py=====分val和train的下标=====
train_part, val_part = val_split(train_dataset_eval)#用train_dataset_tr也行他们两个在dataloader

```

前都一样

```
val_idx = val_part.indices#indices是取相应的下标
train_idx = train_part.indices
#还有一种方式如下：（这种比较麻烦，但也是一个思路）
#val_idx_set = set(val_idx) # set就是把下标清单变成方便做判断的形式
#train_idx = [i for i in range(len(train_dataset_tr)) if i not in val_idx_set] #i in
range(len(train_dataset_tr))其实就是全部的下标

#main.py=====分val和train的dataset=====
train_dataset = Subset(train_dataset_tr, train_idx)#从一个已有的 Dataset 里，按下标挑出一部分，
包成一个新的 Dataset
val_dataset = Subset(train_dataset_eval, val_idx)
print("各数据集的数据量：train", len(train_dataset), "| val", len(val_dataset), "| test",
len(test_dataset))
```

1. 进行数据增强，扩充样本容量（可调库）
2. 调整图片大小以适应CNN的输入：
3. 运用transform将训练集的数据进行镜像，旋转，改亮度，对比度，饱和度等等以完成数据增强
4. 同时运用transform，将数据调整大小为(IMG_SIZE, IMG_SIZE)。但这样会破坏图片比例，还可采取先等比缩放，再裁剪的方法

```
# preprocess.py===== 图像预处理（两种 transform）
=====
# transforms.Compose() 是打包，执行时按列表顺序依次执行
train_tf = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),#还可以Resize(256) + CenterCrop(IMG_SIZE)
    transforms.RandomHorizontalFlip(),# 50% 概率把图左右镜像
    transforms.RandomRotation(20),#随机选一个角度旋转，范围是-20 ~20
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),#调亮度，对比度，饱和度，最多 ±20%
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),#先转张量，再归一（mean和std是公认的常量）
])
```

```
eval_tf = transforms.Compose([#训练集和验证集不进行数据增强
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])
```

IV-炼丹

一. 概念问题

1. 什么是超参数？如何设定？

参数	超参数
进行预测时需要这些参数	估算模型参数时需要这些参数
通过优化算法（梯度下降法、Adam、Adagrad）来估计它们	通过超参数的调优来估计
不是手动设置的	是手动设置的
训练后发现的最终参数将决定该模型在未见过的数据上的表现	超参数的选择决定了训练的效率如何。在梯度下降法中，学习率决定了优化过程中估计参数的效率和准确性。

- 超参数是训练前由人来手动决定的参数（还包括了优化器的选择，也算是超参数），用于其他参数的学习，但超参数本身不随学习训练变化。超参数的设定有两种方法：
- 2. 其一：按照经验去试，就像用了adam后设置lr大多在0.001附近，可以通过多次训练，根据验证集的表现进行调整。
 - 3. 其二：通过算法进行选择，如：网格搜索(Grid Search)，随机搜索(Randomized Search)，贝叶斯优化(Bayesian Optimization)，Hyperband等等（各个方法暂时没有详细了解）
 - 4. 什么是优化器，有哪些类型？

优化器是对参数修正的方法进行优化，通过加入惯性或是自适应学习率等，让模型找到最佳参数组的效率变高，防止掉进局部最小值

优化器	核心机制
SGD	纯靠梯度修正参数

优化器	核心机制
SGD+Momentum	有惯性，冲过浅谷
AdaGrad	累加平方梯度，学习率逐维衰减
RMSProp	减弱梯度大小带来的影响
Adam	动量 + RMSProp，最常用
AdamW	Adam + 解耦权重衰减（要用L2正则时，AdamW比Adam好）

1. DataLoader 与 DataSet是什么及其使用？（可调库）
2. dataset是管理两件事——数据从哪里拿，数据拿哪一个。如果不用dataset计算机根本不知道数据从哪里拿，也不知道怎么拿。dataset想实现的是什么数据都能被用，只要他可以被读取出长度__len__，有——编号__getitem__。
3. dataloader是管理把数据给模型进行训练前的数据处理，他会把数据切成一块一块的，也可以将其中的数据打乱，以便于模型的学习。
4. 值得注意的是dataloader的建立，必须要数据经过了dataset
5. dataset的使用，有时直接调库，有时要自定义（自定义数据或csv数据等）;dataloader始终调库

dataset

```
#调库
dataset = datasets.ImageFolder(root="D:/花卉识别/data", transform=tf)

#自定义
class Mydataset(Dataset):
    def __init__(self, transform=None):
        #设置类里的变量

    def __getitem__(self, idx):
        #当取某一个数据，返回什么
        return img,label

    def __len__(self):
        #数据集的大小
        return len(self.img_path)
```

dataloader

```
train_loader = DataLoader(train, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test, batch_size=batch_size, shuffle=False)
```

二.网络设计

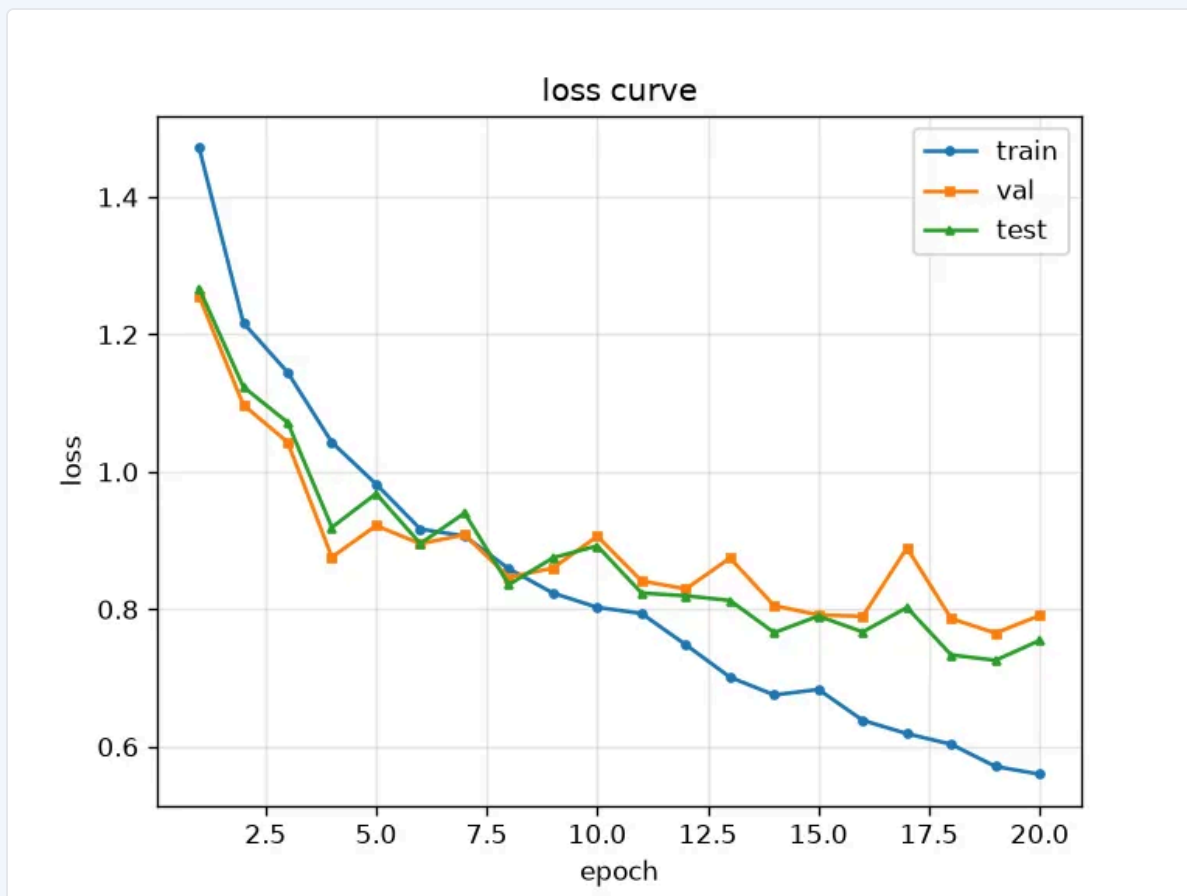
```
class Mymodel(nn.Module):
    def __init__(self, n_classes):
        super(Mymodel, self).__init__()#继承nn.Module必写
        self.conv1 = nn.Conv2d(3, 96, kernel_size=11, stride=4)
        self.conv2 = nn.Conv2d(96, 256, kernel_size=5, padding=2)#kernel_size=5, padding=2让特征图大小不变
        self.conv3 = nn.Conv2d(256, 384, kernel_size=3, padding=1)#kernel_size=3, padding=1让特征图大小不变
        self.conv4 = nn.Conv2d(384, 384, kernel_size=3, padding=1)
        self.conv5 = nn.Conv2d(384, 256, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=3, stride=2)#下一层 = (上一层+1)/2
        self.fc6 = nn.Linear(256 * 6 * 6, 4096)
        self.fc7 = nn.Linear(4096, 4096)#Dense = 全连接层
        self.fc8 = nn.Linear(4096, n_classes)#只有5种花，所以不用图中的18
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.flatten = nn.Flatten()

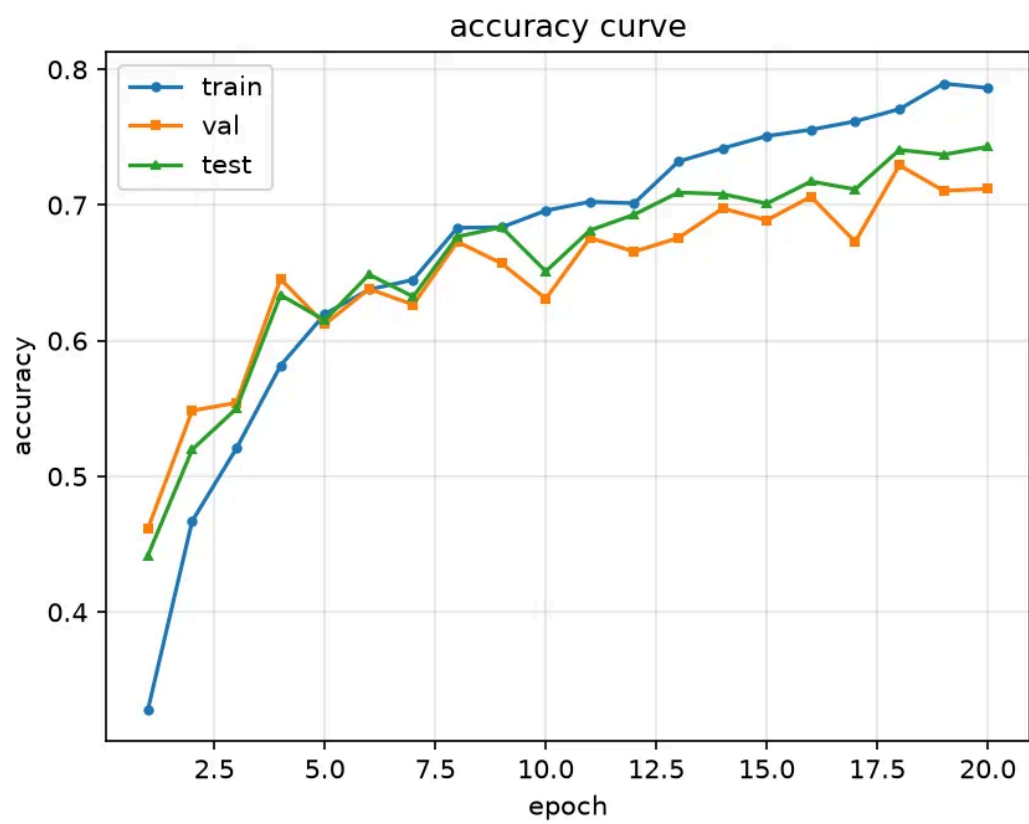
    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = self.relu(self.conv3(x))
        x = self.relu(self.conv4(x))
        x = self.pool(self.relu(self.conv5(x)))
        x = self.flatten(x)#压成一维进全连接层
        x = self.dropout(self.relu(self.fc6(x)))
        x = self.dropout(self.relu(self.fc7(x)))
```

```
x = self.fc8(x)#只有这个不加激活函数，不然会失去表达负数分的能力  
return x
```

三.实验结果

实验结果如下：





```
运行 main x
D:\Anaconda\envs\pytorch\python.exe D:\花卉识别\src\main.py
=====开始工作=====
损坏图片已还原
坏图 13 张
各数据集的数据量: train 2766 | val 691 | test 860
正在使用的设备是: cuda
epoch 1/20 | train loss 1.4813; acc 0.3210 | val loss 1.2648; acc 0.4443 | test loss 1.2786; acc 0.4547
epoch 2/20 | train loss 1.2325; acc 0.4472 | val loss 1.1243; acc 0.4544 | test loss 1.1248; acc 0.4942
epoch 3/20 | train loss 1.1661; acc 0.4920 | val loss 1.0509; acc 0.5745 | test loss 1.0554; acc 0.5767
epoch 4/20 | train loss 1.0997; acc 0.5474 | val loss 1.0678; acc 0.5239 | test loss 1.1108; acc 0.5070
epoch 5/20 | train loss 1.0370; acc 0.5915 | val loss 1.0232; acc 0.5731 | test loss 1.0126; acc 0.5686
epoch 6/20 | train loss 0.9886; acc 0.6135 | val loss 0.9087; acc 0.5991 | test loss 0.9463; acc 0.6314
epoch 7/20 | train loss 0.9304; acc 0.6374 | val loss 0.9054; acc 0.6483 | test loss 0.8486; acc 0.6802
epoch 8/20 | train loss 0.9117; acc 0.6634 | val loss 0.9358; acc 0.6237 | test loss 0.9017; acc 0.6407
epoch 9/20 | train loss 0.8940; acc 0.6565 | val loss 0.8741; acc 0.6585 | test loss 0.8273; acc 0.7023
epoch 10/20 | train loss 0.8439; acc 0.6844 | val loss 0.8367; acc 0.6570 | test loss 0.8296; acc 0.6860
epoch 11/20 | train loss 0.7802; acc 0.7068 | val loss 0.8322; acc 0.6860 | test loss 0.8426; acc 0.7047
epoch 12/20 | train loss 0.7983; acc 0.6945 | val loss 0.8287; acc 0.6831 | test loss 0.8174; acc 0.6860
epoch 13/20 | train loss 0.7662; acc 0.7198 | val loss 0.7909; acc 0.6889 | test loss 0.7820; acc 0.7174
epoch 14/20 | train loss 0.7204; acc 0.7256 | val loss 0.7979; acc 0.6671 | test loss 0.8463; acc 0.6802
epoch 15/20 | train loss 0.7302; acc 0.7285 | val loss 0.8286; acc 0.6946 | test loss 0.7647; acc 0.7279
epoch 16/20 | train loss 0.6925; acc 0.7440 | val loss 0.7401; acc 0.7192 | test loss 0.7287; acc 0.7244
epoch 17/20 | train loss 0.6690; acc 0.7495 | val loss 0.7426; acc 0.7062 | test loss 0.7621; acc 0.7267
epoch 18/20 | train loss 0.6196; acc 0.7715 | val loss 0.6928; acc 0.7279 | test loss 0.6789; acc 0.7593
epoch 19/20 | train loss 0.6086; acc 0.7650 | val loss 0.7636; acc 0.7091 | test loss 0.7586; acc 0.7151
epoch 20/20 | train loss 0.5870; acc 0.7758 | val loss 0.7751; acc 0.7004 | test loss 0.7508; acc 0.7349
最高验证集准确率 0.7279
模型已保存 D:\花卉识别\logs\best.pth
loss_curve 已保存: D:\花卉识别\docx\loss_curve
accuracy_curve 已保存: D:\花卉识别\docx\accuracy_curve
已完成所有任务 (恭喜恭喜恭喜恭喜~)
```

其他收获

1. 关于dataset中函数的调用:

你写的代码	Python 实际调用
<code>ants_dataset[0]</code>	<code>ants_dataset.__getitem__(0)</code>
<code>len(ants_dataset)</code>	<code>ants_dataset.__len__()</code>

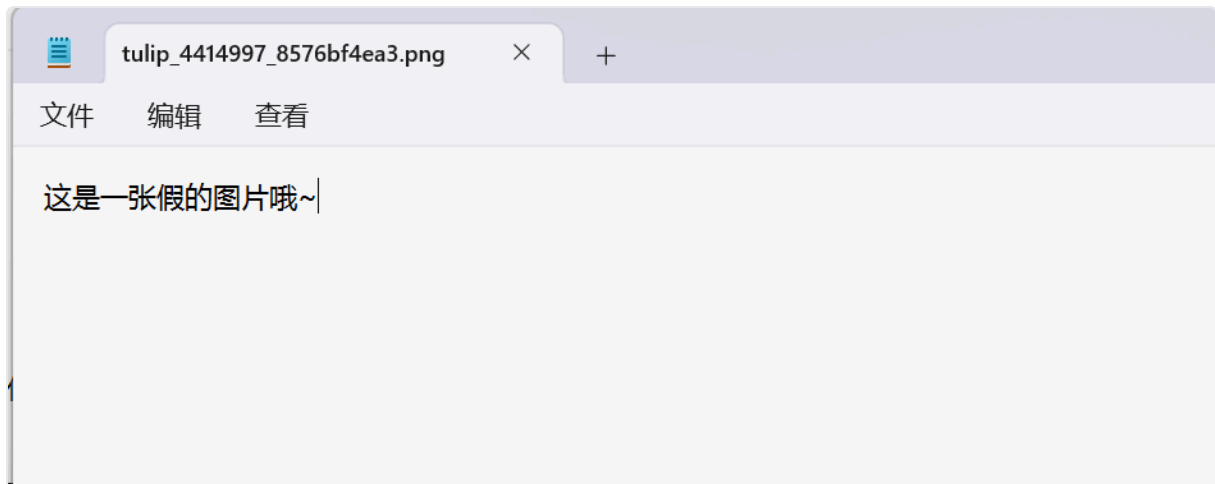
2. 模型构建中用的公式: 输出尺寸 = (输入尺寸 - 核尺寸 + 2×padding) / stride + 1

疑惑

1. 在一轮中的此题，有一题问的是卷积神经网络和全连接神经网络中的卷积操作有区别吗？
 - 关于全连接神经网络中的卷积操作，其实我查了CSDN等博客，Google，AI等等，其实我没有找到很明确的说明全连接神经网络中有卷积操作，有说法是全连接层中的线性处理是卷积操作的退化，我觉得有点牵强，其实也有合理的地方：卷积层中的卷积操作就是把卷积核（权重）输入的二阶张量+*bias*（偏置），和线性的 $WX+B$ 有相似之处，如果按这个说法，CNN中的卷积操作是以同一个卷积核和偏置以输入数据的一部分做计算，一步一步计算完整个数据；而MLP中的卷积操作是一整个数据直接计算，且参数随训练变化。还有一种可能这里的卷积操作，是指处于卷积神经网络中，全连接神经网络前的卷积层的操作，是卷积神经网络的组成部分之一。了解到有dataset属性这一说法，

小发现/小巧思

1. 在处理坏图的过程中，发现.png并不是图片，而是文本文件，用记事本打开后，里面有一句小彩蛋！！



2. 在构造model时发现，如果按照图中的224去构造，是无法完成的（除不尽），所以改成227；图中最终的输出值为18，我认为分几类输出值就为几，所以改为5
3. 为了更好的反复看到清理坏图的效果，设置了一个坏图还原函数，可通过改变config中的RESTORE来控制是否进行坏图复原.

部分资料

[CNN感受野全解析](#) | [深度学习动画_哔哩哔哩_bilibili](#)

[卷积神经网络基础—全连接层、卷积层、激活层、池化层_卷积层 激活层-CSDN博客](#)

[超参数调优全攻略-CSDN博客](#)

[吴恩达Pytorch深度学习教程](#)